

Tugas Implementasi: Real-time Remote Photoplethysmography (rPPG)

Mata Kuliah: Sistem Teknologi Multimedia

Nama: Fawwaz Abhitah Sugiarto

NIM: 122140014

Import library yang dibutuhkan

```
In [1]: # Library yang akan dipakai
import cv2
import numpy as np
import time
import mediapipe as mp
from collections import deque
from scipy.signal import butter, filtfilt, find_peaks
from scipy.ndimage import uniform_filter1d
import matplotlib.pyplot as plt

# Cek versi library utama
print("OpenCV :", cv2.__version__)
print("NumPy   :", np.__version__)
print("MediaPipe :", mp.__version__)
```

OpenCV : 4.11.0
NumPy : 1.26.4
MediaPipe : 0.10.21

Konfigurasi parameter rPPG

```
In [ ]: # Pengaturan awal untuk proses rPPG
frame_rate = 30          # Target FPS kamera yang digunakan
buffer_size = 300        # Jumlah frame yang disimpan dalam window
freq_low = 0.67          # bandpass filter 0.67 Hz (40 BPM)
freq_high = 4.0          # bandpass filter 4.0 Hz (240 BPM)

# Inisialisasi modul deteksi wajah dari MediaPipe
mp_face = mp.solutions.face_detection
detektor_wajah = mp_face.FaceDetection(model_selection=0,min_detection_confidence=0.5)
```

Filter Sinyal

```
In [4]: # Filter bandpass untuk membersihkan sinyal rPPG
def saring_sinyal(input_sinyal, fs=frame_rate, low=freq_low, high=freq_high, orde=4):
    # normalisasi cutoff terhadap frekuensi Nyquist
    batas_nyquist = fs * 0.5
    low_norm = low / batas_nyquist
    high_norm = high / batas_nyquist

    # koefisien filter butterworth
    b, a = butter(orde, [low_norm, high_norm], btype='band')
```

```

# terapkan filter secara forward-backward
hasil = filtfilt(b, a, input_sinyal)
return hasil

# Menghitung BPM menggunakan sinyal rPPG yang telah difilter
def hitung_bpm(sinyal, fs=frame_rate, low=freq_low, high=freq_high):
    # pastikan panjang sinyal cukup
    if len(sinyal) < fs * 3:
        return 0.0, np.zeros_like(sinyal)

    # hilangkan nilai rata-rata
    raw = np.array(sinyal)
    tanpa_dc = raw - raw.mean()

    # Lakukan filtering
    terfilter = saring_sinyal(tanpa_dc, fs, low, high)

    # smoothing agar puncak lebih stabil
    smoothed = uniform_filter1d(terfilter, size=5)

    # cari puncak dengan jarak minimal antar puncak
    min_jarak = int(fs * 0.3)
    puncak, _ = find_peaks(smoothed, distance=min_jarak)

    # hitung BPM berdasarkan selisih antar puncak
    if len(puncak) > 1:
        selisih_detik = np.mean(np.diff(puncak)) / fs
        bpm = 60.0 / selisih_detik
    else:
        bpm = 0.0

    return bpm, smoothed

```

Penggunaan POS method

```

In [5]: def metode_pos(buffer_rgb):
    # ubah buffer menjadi array (N x 3)
    matriks_rgb = np.array(buffer_rgb, dtype=float)

    # normalisasi berdasarkan rata-rata tiap kanal
    mean_rgb = matriks_rgb.mean(axis=0)
    mean_rgb[mean_rgb == 0] = 1 # mencegah pembagian nol
    norm_rgb = (matriks_rgb / mean_rgb) - 1

    # pisahkan tiap kanal
    r = norm_rgb[:, 0]
    g = norm_rgb[:, 1]
    b = norm_rgb[:, 2]

    # dua proyeksi sinyal POS
    sinyal1 = g - b
    sinyal2 = g + b - 2 * r

    # sesuaikan proporsi energi kedua sinyal
    if np.std(sinyal2) != 0:
        alpha = np.std(sinyal1) / np.std(sinyal2)
    else:

```

```

alpha = 1.0

# hasil kombinasi POS
hasil_pos = sinyal1 + alpha * sinyal2

return hasil_pos

```

Implementasi Real-time rPPG

```

In [6]: # Buffer untuk menyimpan nilai RGB dan kanal hijau
rekam_rgb = deque(maxlen=buffer_size)
rekam_hijau = deque(maxlen=buffer_size)

# Variabel tampilan BPM
bpm_green = 0.0
bpm_pos = 0.0
waktu_terakhir = time.time()

# Inisialisasi kamera
kamera = cv2.VideoCapture(0)
print("Tekan ESC untuk keluar.")

try:
    while kamera.isOpened():
        berhasil, frame = kamera.read()
        if not berhasil:
            break

        # ubah ke RGB untuk MediaPipe
        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

        # deteksi wajah
        hasil = detektor_wajah.process(frame_rgb)
        ada_roi = False

        if hasil.detections:
            for det in hasil.detections:
                lokasi = det.location_data.relative_bounding_box
                h, w, _ = frame.shape

                x = int(lokasi.xmin * w)
                y = int(lokasi.ymin * h)
                bw = int(lokasi.width * w)
                bh = int(lokasi.height * h)

                # pastikan ROI valid
                if x >= 0 and y >= 0 and x + bw <= w and y + bh <= h:
                    roi = frame_rgb[y:y+bh, x:x+bw]

                    if roi.size > 0:
                        nilai = np.mean(roi, axis=(0, 1)) # [R, G, B]
                        rekam_rgb.append(nilai)
                        rekam_hijau.append(nilai[1])
                        ada_roi = True

            # tampilkan bounding box
            cv2.rectangle(frame, (x, y), (x+bw, y+bh), (0, 255, 0),
                          cv2.putText(frame, "Wajah", (x, y - 5),
                                          cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0),

```

```

# update BPM setiap 1 detik
if ada_roi and len(rekam_hijau) > frame_rate * 2:
    if time.time() - waktu_terakhir >= 1:
        bpm_green, _ = hitung_bpm(list(rekam_hijau))

        sinyal_pos = metode_pos(list(rekam_rgb))
        bpm_pos, _ = hitung_bpm(sinyal_pos)

        waktu_terakhir = time.time()

# tampilkan BPM di frame
cv2.putText(frame, f"BPM Hijau : {bpm_green:.1f}", (20, 40),
            cv2.FONT_HERSHEY_SIMPLEX, 0.75, (0, 255, 0), 2)

cv2.putText(frame, f"BPM POS : {bpm_pos:.1f}", (20, 75),
            cv2.FONT_HERSHEY_SIMPLEX, 0.75, (0, 255, 255), 2)

cv2.imshow("rPPG Realtime", frame)

# tombol ESC untuk berhenti
if cv2.waitKey(1) == 27:
    break

finally:
    kamera.release()
    cv2.destroyAllWindows()
    print("Proses dihentikan.")

```

Tekan ESC untuk keluar.
Proses dihentikan.

Visualisasi sinyal

```

In [7]: # Visualisasi sinyal terakhir dalam buffer
if len(rekam_rgb) > frame_rate * 2:

    # ambil snapshot buffer
    data_hijau = list(rekam_hijau)
    data_pos_raw = metode_pos(list(rekam_rgb))

    # proses ulang dengan filter agar hasil curvanya bersih
    bpm_hijau, sinyal_hijau = hitung_bpm(data_hijau)
    bpm_pos, sinyal_pos = hitung_bpm(data_pos_raw)

    # mulai plotting
    plt.figure(figsize=(12, 7))

    # grafik kanal hijau
    plt.subplot(2, 1, 1)
    plt.plot(sinyal_hijau, color="green", label="Filtered Green Signal")
    plt.title(f"Sinyal rPPG - Kanal Hijau | BPM: {bpm_hijau:.1f}")
    plt.ylabel("Amplitudo")
    plt.grid(alpha=0.35, linestyle="--")
    plt.legend()

    # grafik POS
    plt.subplot(2, 1, 2)
    plt.plot(sinyal_pos, color="orange", label="POS Signal (Filtered)")

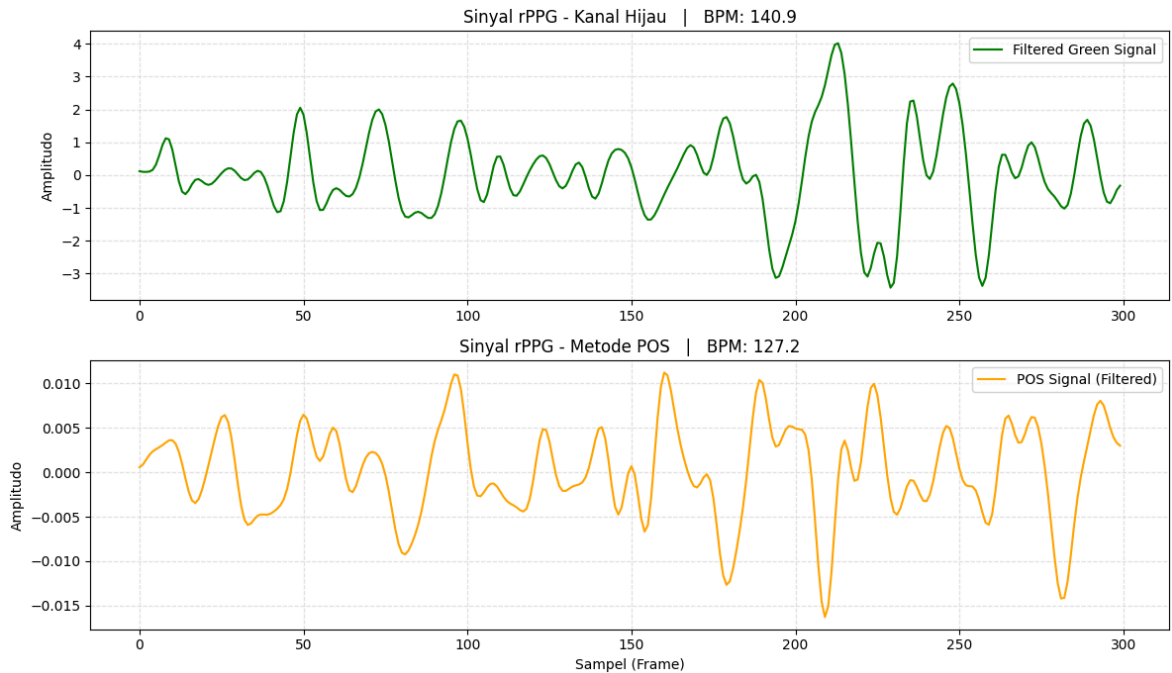
```

```
plt.title(f"Sinyal rPPG - Metode POS | BPM: {bpm_pos:.1f}")
plt.xlabel("Sampel (Frame)")
plt.ylabel("Amplitudo")
plt.grid(alpha=0.35, linestyle="--")
plt.legend()

plt.tight_layout()
plt.show()
```

else:

```
print("Data buffer belum cukup untuk menampilkan grafik.")
```



Penjelasan

1. Deteksi Wajah dan Ekstraksi Sinyal

Program menggunakan MediaPipe Face Detection untuk mengidentifikasi keberadaan wajah pada setiap frame yang ditangkap kamera. Setelah wajah terdeteksi, area tersebut digunakan untuk menghitung rata-rata nilai **R**, **G**, dan **B** sebagai representasi perubahan warna kulit. Kanal **Hijau** dipilih sebagai dasar sinyal rPPG karena menunjukkan respon paling jelas terhadap perubahan aliran darah dibandingkan kanal warna lainnya.

2. Pemrosesan Sinyal

Setelah sinyal hijau terkumpul, beberapa langkah pemrosesan dilakukan untuk memperjelas pola detak jantung:

- **Detrending:** menghapus komponen rata-rata agar fluktuasi kecil lebih terlihat.
- **Bandpass Filter (0.67–4.0 Hz):** menjaga sinyal dalam rentang frekuensi detak jantung manusia dan mereduksi noise.
- **Smoothing:** memberikan perataan tambahan agar puncak sinyal tidak terlalu berisik.
- **Peak Detection:** mencari puncak berulang pada sinyal terfilter dan mengubahnya menjadi nilai BPM.

Tahapan ini membuat sinyal kasar dari kamera menjadi lebih terstruktur sehingga dapat dihitung secara akurat.

3. Metode POS (Perbaikan Akurasi)

Sebagai upaya untuk memperoleh sinyal yang lebih stabil, terutama ketika terdapat variasi pencahayaan atau gerakan kecil, sistem juga menerapkan metode **POS (Plane-Orthogonal-to-Skin)**. Pendekatan ini bekerja dengan menormalkan nilai RGB, kemudian mengkombinasikan ketiganya menjadi dua sinyal baru yang lebih stabil terhadap perubahan intensitas cahaya. Hasil proyeksi POS menghasilkan pola gelombang yang lebih rapi dan konsisten, sehingga proses perhitungan BPM menjadi lebih akurat dibandingkan hanya menggunakan kanal hijau.

4. Perbandingan Hasil Green Channel dan POS

Berdasarkan karakter sinyal:

- **Green Channel:** amplitudo besar dan mudah terbaca, namun sensitif terhadap perubahan intensitas cahaya dan noise gerakan kecil.
- **POS Method:** sinyal lebih stabil, fluktuasi lebih terkontrol, dan umumnya memberikan estimasi BPM yang lebih konsisten.

Visualisasi grafik menunjukkan bahwa sinyal POS cenderung lebih bersih dibandingkan kanal hijau.

5. Proses Real-Time

Pada mode real-time, program memanfaatkan buffer sepanjang 300 frame (sekitar 10 detik pada 30 FPS) untuk menyimpan sinyal RGB secara berkelanjutan. Data dalam buffer diperbarui setiap frame, dan estimasi BPM dihitung secara periodik berdasarkan sinyal yang telah melalui proses filtering. Nilai BPM dapat berubah secara responsif sambil tetap mempertahankan kestabilan karena dihitung menggunakan rentang data yang cukup panjang untuk membentuk pola detak jantung yang jelas.